

Операционные системы

Внешний курс. Работа с vim, bash, sed, find, gnuplot

Кара Егор Андреевич

2026-01-05

- 1 Информация
- 2 Цель работы
- 3 vim
- 4 Процессы и оболочки
- 5 bash-скрипты
- 6 Условия в bash
- 7 Арифметика и функции
- 8 grep, sed, find
- 9 gnuplot
- 10 Права доступа
- 11 Вывод

Раздел 1

Информация

- Кара Егор Андреевич



- Кара Егор Андреевич
- НБИбд-01-25, студент



- Кара Егор Андреевич
- НБИбд-01-25, студент
- Российский университет дружбы народов им. П. Лумумбы



- Кара Егор Андреевич
- НБИбд-01-25, студент
- Российский университет дружбы народов им. П. Лумумбы
- 1032253851@rudn.ru



Раздел 2

Цель работы

Целью работы является освоение продвинутых инструментов Linux: редактора vim, написания bash-скриптов, работы с условиями и циклами, использования sed и grep, поиска файлов через find, а также построения графиков в gnuplot.

Раздел 3

vim

Перемещение по словам

Команды **w** и **W** перемещают курсор по словам, но учитывают разделители по-разному.

После 10 нажатий на **W** курсор окажется в том же месте, что и после 10 нажатий **w**, если текст не содержит специальных символов.

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (**w**, **e**, **b**) или большую (**W**, **E**, **B**) букву. Первые перемещают нас по "словам" (**word**), а вторые по "большим словам" (**WORD**). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между **word** и **WORD**.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению **w**, нужно открыть vim и ввести команду **:help w**. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про **e** и про **b** и, самое главное, про **word** и **WORD**. Кроме того, можно вызвать сразу справку по термину **word** при помощи **:help word**. Чтобы закрыть справку, нужно ввести команду **:q**.

Выберите все подходящие ответы из списка



Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

Комбинации для редактирования

Команды `d2w$bifour<Esc>`, `ddithree four four four five<Esc>` и `d2wwywPp` выполняют корректное редактирование текста, используя удаление, вставку и копирование. Это демонстрирует сочетание нормального, вставочного и операторного режимов vim.

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже **наборов нажатий клавиш** выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как `<Esc>` (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка



Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

Замена слов в файле

Команда `:%s/Windows/Linux/` заменяет первое вхождение слова `Windows` в каждой строке файла на `Linux`.

Это пример глобальной замены с ограничением на одно совпадение в строке.

Предположим, что вы открыли файл в редакторе `vim` и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в `vim`? Укажите необходимую команду целиком (т.е. **включая** ввод `:` в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст



Верно.

```
:%s/Windows/Linux/
```

Visual-режим

Режим выделения открывается клавишей `v`,

внизу появляется надпись `-- VISUAL --`.

В этом режиме можно удалять (`d`) и копировать (`y`) выделенный текст.

Выход — дважды нажать `Esc`.

Мы совсем не рассказали вам про третий режим работы `vim` — режим **выделения** (**Visual**). Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в `vimtutor`, который мы настоятельно рекомендуем вам для изучения `vim`!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить `vimtutor` целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка



Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

Раздел 4

Процессы и оболочки

История команд

При запуске нового shell создаётся новый процесс.

История команд хранится отдельно для каждого процесса, поэтому при перемещении стрелками вверх/вниз отображаются только команды текущего shell.

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка



Хорошая работа.

- ☐ Только из набора А
- ☐ Никакие команды появляться не будут

Раздел 5

bash-скрипты

Абсолютный путь

После выполнения скрипта файл `file1.txt` окажется в `/home/bi/file1.txt`, так как он создаётся в домашней директории перед переходом в Desktop.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка



Всё правильно.

☒ `/home/bi/file1.txt`

Имена переменных

Корректные имена переменных:

`__variable`, `VARiable`, `variable123`, `_variable`, `variable_123`.

Имя не может начинаться с цифры.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка



Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ `vari/able`

☒ `_variable`

Скрипт с аргументами

Скрипт принимает два аргумента и выводит их значения.

Важно экранировать `$` внутри строки, чтобы избежать подстановки.

Напишите скрипт на `bash`, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`



Так точно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 var1=$1
2 var2=$2
```

Раздел 6

Условия в bash

Условия, всегда возвращающие True

Примеры выражений, которые всегда истинны:

```
$var1 == $var2 || $var1 != $var2,
```

```
$# -ge 0,
```

```
-e $0,
```

```
5 -ge 5,
```

```
-z "".
```

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `'...'` (внутри `[[ ]]` и не забудьте про пробелы после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что

В примере со значениями `var=3` и `var=5`

ни одно условие не выполняется, поэтому выводится **"four"** в обоих случаях.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

Скрипт о количестве студентов

Скрипт корректно обрабатывает диапазоны значений:

1 студент, 2–4 студента, 5 и более — «A lot of students».

Напишите скрипт на `bash`, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "→" выводить не нужно.

Примечание б): в последней строке слова "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверить вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`



Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2     echo "1 student"
```

Циклы

Скрипт выводит 5 раз "start" и 4 раза "finish",
так как **finish** печатается после итерации цикла.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "**start**", а сколько раз слово "**finish**"?

Выберите один вариант из списка



Отлично!

Определение возрастной группы

Скрипт использует вложенные условия для классификации возраста.

Это пример структурированного ветвления.

Напишите программу. Тестируется через stdin → stdout



Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

```
1 while [[ 1==1 ]]
2 do
3     group=""
4     echo "enter your name:"
5     read name
6     if [[ -z $name ]]
7     then
8         break
9     fi
10    echo "enter your age:"
11    read age
12    if [[ $age -eq 0 ]]
13    then
14        break
15    fi
16    if [[ $age -le 16 ]]
17    then
18        group="child"
19    elif [[ $age -le 25 ]]
```

Раздел 7

Арифметика и функции

Увеличение переменной

Команды, увеличивающие a на значение b , используют `let, (( )),` или `a=$((a+b))`.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b` ? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка



Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

После 10 вызовов функции `counter`

переменная `c1` пуста, а `c2` равна 110 — удвоенной сумме 1..10.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из `bash`-скрипта:

```
counter () # takes one argument
{
  local let "c1+=$1"
  let "c2+=${1}*2"
}
```

Впишите в форму ниже строку, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте после десяти вызовов функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на `bash`, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст



Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свой

НОД двух чисел

Реализация алгоритма Евклида в bash

позволяет вычислить наибольший общий делитель.

Напишите программу. Тестируется через stdin → stdout



Верно. Так держать!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 while [ true ]
3 do
4     read n1 n2
5     if [ -z $n1 ]; then
6         echo "bye"
7         break
8     else
9         gcd () {
10             remainder=1
11             if [ $n2 -eq 0 ]
12             then
13                 echo "bye"
14             fi
15             while [ $remainder -ne 0 ]
16             do
17                 remainder=$((n1%n2))
18                 n1=$n2
19                 n2=$remainder
20             done
21         }
```

Скрипт выполняет арифметические операции,
используя конструкцию `case`.

Напишите программу. Тестируется через `stdin` → `stdout`



Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 #!/bin/bash
3 while [[ True ]]
4 do
5     read birinchi amal ikkinchi
6     if [[ $birinchi == "exit" ]]
7     then
8         echo "bye"
9         break
10    elif [[ "$birinchi" =~ "^[0-9]+$" && "$ikkinchi" =~ "^[0-9]+$" ]]
11    then
12        echo "error"
13        break
14    else
15        case $amal in
16            "+") let "result = birinchi + ikkinchi";;
17            "-") let "result = birinchi - ikkinchi";;
18            "/" let "result = birinchi / ikkinchi";;
19            "*" let "result = birinchi * ikkinchi";;
20            "%" let "result = birinchi % ikkinchi";;
21            "**") let "result = birinchi ** ikkinchi";;
22            *) echo "error" ; break ;;
```

Раздел 8

grep, sed, find

Команда `find -iname "star*"` найдёт файлы независимо от регистра, в отличие от `-name`.

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek_OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ** **найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка



Хорошие новости, верно!



☒ `Star_Wars.avi`



☐ `star_trek_OST.mp3`



☐ `stardust.mpeg`



☒ `STARS.txt`



Регулярные выражения

Команда `grep -E "[xklXKL]?[uU]buntu$"`

находит строки, оканчивающиеся на Ubuntu с необязательным префиксом.

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xklXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка



Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☐ Lubuntu is better than Windows
- ☐ Uuuubuntu!
- ☒ Hmm, XKLubuntu
- ☒ Lubuntu is better than Ubuntu

Без `-n` sed выводит строки дважды:
оригинал + результат обработки.

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка



Отличное решение!

- ☒ Каждая строчка будет выведена два раза
- ☐ Будут выведены все строки файла text.txt, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла text.txt
- ☐ Появится сообщение об ошибке

Замена аббревиатур

Команда `sed` заменяет все аббревиатуры на слово `"abbreviation"` и записывает результат в `edited.txt`.

Напишите текст



Верно. Так держать!

```
sed -r 's/[A-Z]{2,} / abbreviation /g' input.txt > edited.txt
```

Раздел 9

gnuplot

Опция persist

Флаг `-p` сохраняет графики после выхода из `gnuplot`.

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

- ☒ Хорошая работа.
- ☐ `-s, --show-plots-after-exit`
- ☒ `-p, --persist`
- ☐ Такой опции не существует
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!

Следующий шаг

Решить снова

Построение графиков

Название ряда — первое значение второго столбца,
точек — 9, так как первая строка пропущена.

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead  
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название у построенного ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка



Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).



Название -- первое значение из первого столбца, нарисовано 10 точек



Название "rowname" нарисовано 10 точек

Изменение move.rot

Изменены инструкции для:

- зеркального отражения по оси Z,
- обратного направления вращения,
- удвоения скорости.

Напишите текст



Верно.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Раздел 10

Права доступа

chmod и chown

Команды позволяют установить права `rw-rw-r--`

и объясняют, почему пользователь из группы всё ещё может создавать файлы.

Выберите все подходящие ответы из списка



Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chown user:group dir`
- ☒ `sudo chmod o+w dir`
- ☐ `chown user:group dir`
- ☒ `sudo chown user dir`
- ☐ `sudo chmod o+x dir`
- ☒ `sudo chmod a+w dir`

Следующий шаг

Решить снова

Команда `wc` считает строки, слова, байты и символы.

Отметьте какие характеристики файла можно посчитать с использованием команды `wc` .

Выберите все подходящие ответы из списка



Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Количество предложений
- ☒ Количество символов
- ☒ Количество строк
- ☒ Количество слов
- ☐ Количество определенных букв (например, количество букв "А")

Следующий шаг

Решить снова

Размер директории

Команда `du -sh` . выводит размер текущей директории в удобном формате.

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо `2048 байт` надо выводить `2.0K`) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт` , то загаданная команда должна вывести на экран одно число: `2.4M` (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно *текущей* директории).

Напишите текст



Хорошие новости, верно!

```
du -h -s
```

Создание директорий

Минимальная команда для создания `dir1`, `dir2`, `dir3`:

```
mkdir dir{1..3}.
```

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст



Так точно!

```
mkdir dir{1..3}
```

Раздел 11

Вывод

В ходе работы были освоены продвинутые инструменты Linux: vim, bash-скрипты, условия и циклы, sed, grep, find, gnuplot. Получены навыки, необходимые для автоматизации, анализа данных и эффективной работы в терминале.